

Tricks of the trade

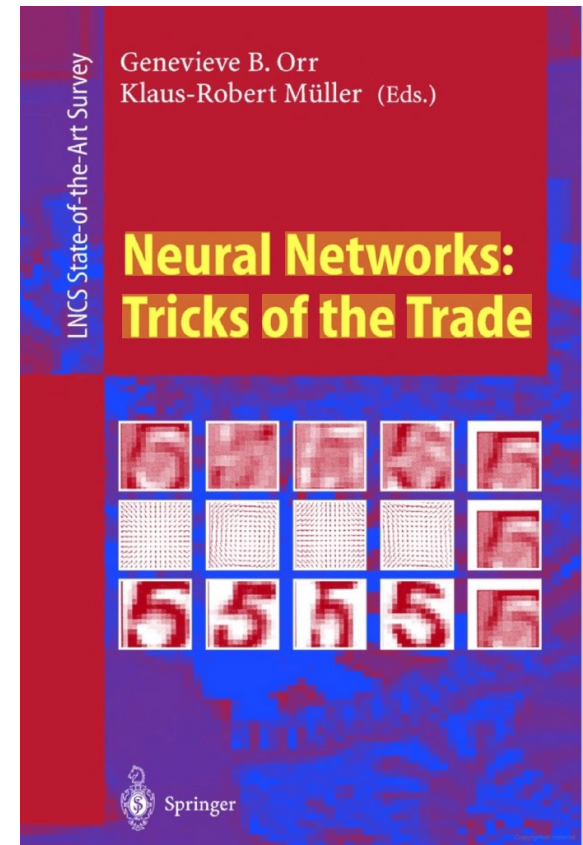
Garrison W. Cottrell

Gary's Unbelievable Research Unit (GURU)

UCSD



Reading for this lecture is
Lecun98efficient.pdf under Readings.
This lecture is based on:
Yann LeCun, Leon Bottou. Genevieve Orr,
Klaus Robert-Müller (1998). Efficient
Backprop. In *Neural Networks: Tricks of the
Trade* (GB Orr & KR Müller, Eds.) Springer:
Berlin.
[LET ME KNOW IF IT ISN'T THERE!]



This lecture

- Tricks of the Trade:
 - These “tricks” are somewhat ancient at this point
 - BUT, they illustrate a lot of good points/issues, some of which have been “solved”, or at least eliminated as issues.
 - These ideas led to “batch normalization” which has made training deep nets much easier

This lecture

- Tricks of the Trade:
 - Stochastic gradient descent versus batch learning
 - Shuffling the examples
 - Perform PCA of the inputs
 - Change the sigmoid
 - Initialize the weights
 - Momentum
 - Adaptive learning rates

This lecture

- Tricks of the Trade:
 - *Stochastic gradient descent versus batch learning*
 - Shuffling the examples
 - Perform PCA of the inputs
 - Change the sigmoid
 - Initialize the weights
 - Momentum
 - Adaptive learning rates

Batch Gradient Descent

- This is a learning method where:
 1. Get one example
 2. Compute the gradient
 3. Add it to a running average of the gradients
 4. If you have seen all of the examples, now change the weights
 5. Go to 1.
- This is following the true gradient for the error function
- You only update the weights once per *epoch* – one pass through the entire training set.

Stochastic Gradient Descent (SGD)

- This is an *online* learning method:
 1. Get one example
 2. Compute the gradient
 3. Change the weights
 4. If you have seen all of the examples, reshuffle them
 5. Goto 1.
- It is *stochastic* because after you have gone through all of the examples (called one *epoch*), the examples are re-shuffled – so the actual gradients you follow are in random order
- And the gradients are computed for each example – so sometimes they can get around local minima, because the model is not following the true (total) gradient.

Why Stochastic gradient descent?

1. It tends to learn faster due to redundancy in the training set – for example, MNIST.
 - You can learn a lot about the problem well before you have seen all of the examples
2. It tends to get better solutions, and better generalization
3. It can adapt to a changing environment
4. It can be used easily for VERY large datasets due to its online nature.

Why Batch learning?

1. Convergence conditions are better understood – in general, there's a fair amount of theory.
2. Better optimization techniques exist, and there are 2nd order techniques that use the 2nd derivative as well (Conjugate gradient, BFGS, etc. – *but*, since 2012, all sorts of new online approaches have been developed).
3. But this aggressive optimization can lead to large weights – with poorer generalization.

Why Mini-Batch learning?

- Again, often for real problems, there is a lot of *redundancy* in the training set.
- The gradient for the first half of the training set is nearly identical to that of the second half.
- A useful compromise between SGD and batch learning is to use a “mini-batch” – say, 128 examples.
- Compute the gradient for each example using the same weights (like batch learning).
- Change the weights once.
- This can be very efficient on GPUs, using matrix-matrix multiplication – a matrix of 128 inputs, times the weights...
- This is *still* called “stochastic gradient descent” SGD because it still has a stochastic component.

This lecture

- Tricks of the Trade:
 - Stochastic gradient descent versus batch learning
 - *Shuffling the examples*
 - Perform PCA of the inputs
 - Change the sigmoid
 - Initialize the weights
 - Adaptive learning rates
 - Momentum

Shuffle the examples (SGD)

1. In particular, shuffling means that we will see patterns from different classes.
2. It is important that a minibatch have most of the categories in it, so the model learns to discriminate the classes
3. A possible heuristic: present examples with more error more often – but *use with caution!* Sometimes the hard examples are hard because:
 - a) there is an error in the training set – a pattern is incorrectly labeled!
 - b) more has to be learned about the easy examples before the hard ones can be tackled
 - c) but, this approach can improve performance on infrequent examples, e.g., /z/ in phoneme recognition.

This lecture

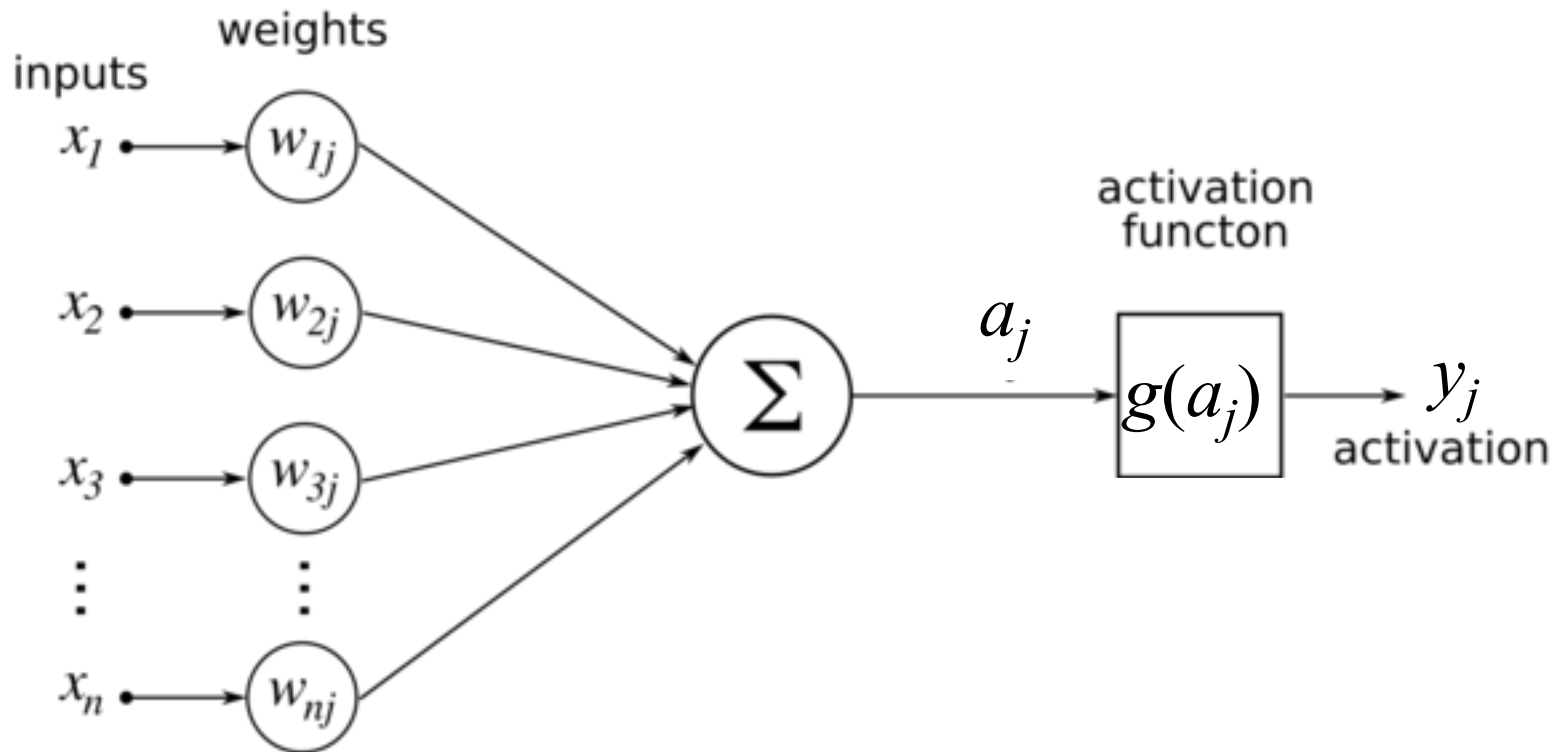
- Tricks of the Trade:
 - Stochastic gradient descent versus batch learning
 - Shuffling the examples
 - *Perform PCA of the inputs*
 - Change the sigmoid
 - Initialize the weights
 - Adaptive learning rates
 - Momentum

This lecture

- Tricks of the Trade:
 - Stochastic gradient descent versus batch learning
 - Shuffling the examples
 - *Perform PCA of the inputs: But first, some intuition...*
 - Change the sigmoid
 - Initialize the weights
 - Momentum
 - Adaptive learning rates

What's wrong with all positive inputs?

What happens to the weight changes for all of the *incoming weights* to one hidden unit, call it j ? Here, all the x_i 's are positive:



Clicker Question

In stochastic gradient descent (SGD), we should shuffle our minibatch before computing its gradients.

A. True

B. False

Clicker Question

In stochastic gradient descent (SGD), we should shuffle our minibatch before computing its gradients.

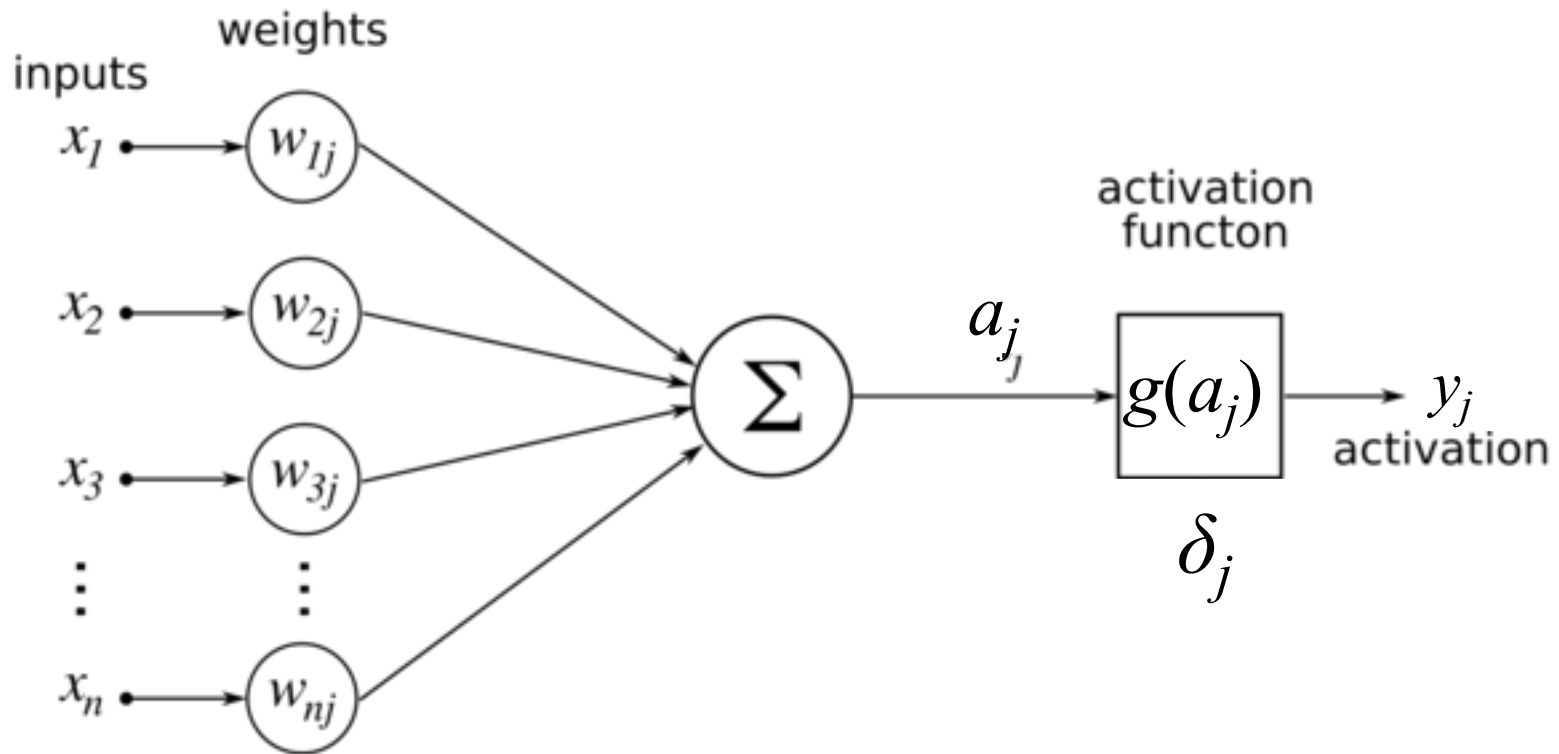
A. True

B. False

(Shuffling now has no impact on the weight changes, you need to shuffle the data before partitioning it into mini-batches!)

So I ask again, What's wrong with all positive inputs?

Recall the weight change rule is: $w_{ij} = w_{ij} + \alpha \delta_j x_i$



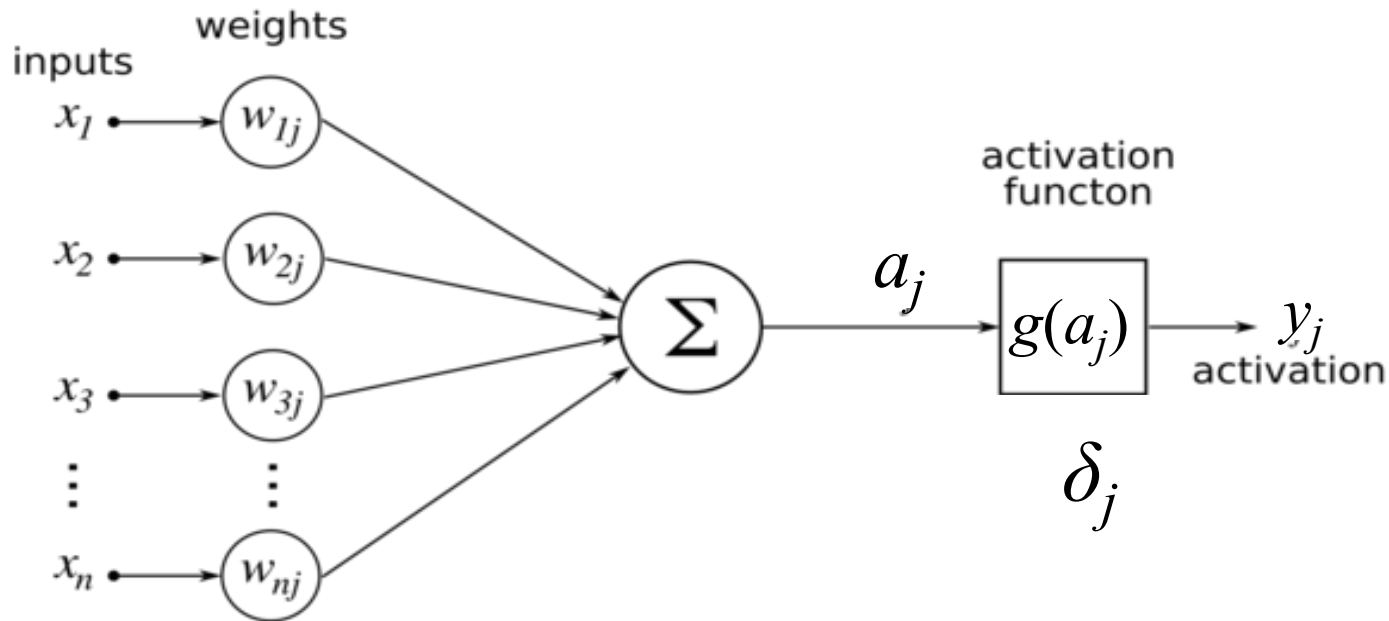
What's wrong with all positive inputs?

The weight change rule is: $w_{ij} = w_{ij} + \alpha \delta_j x_i$

δ_j is the same sign for all of these weights.

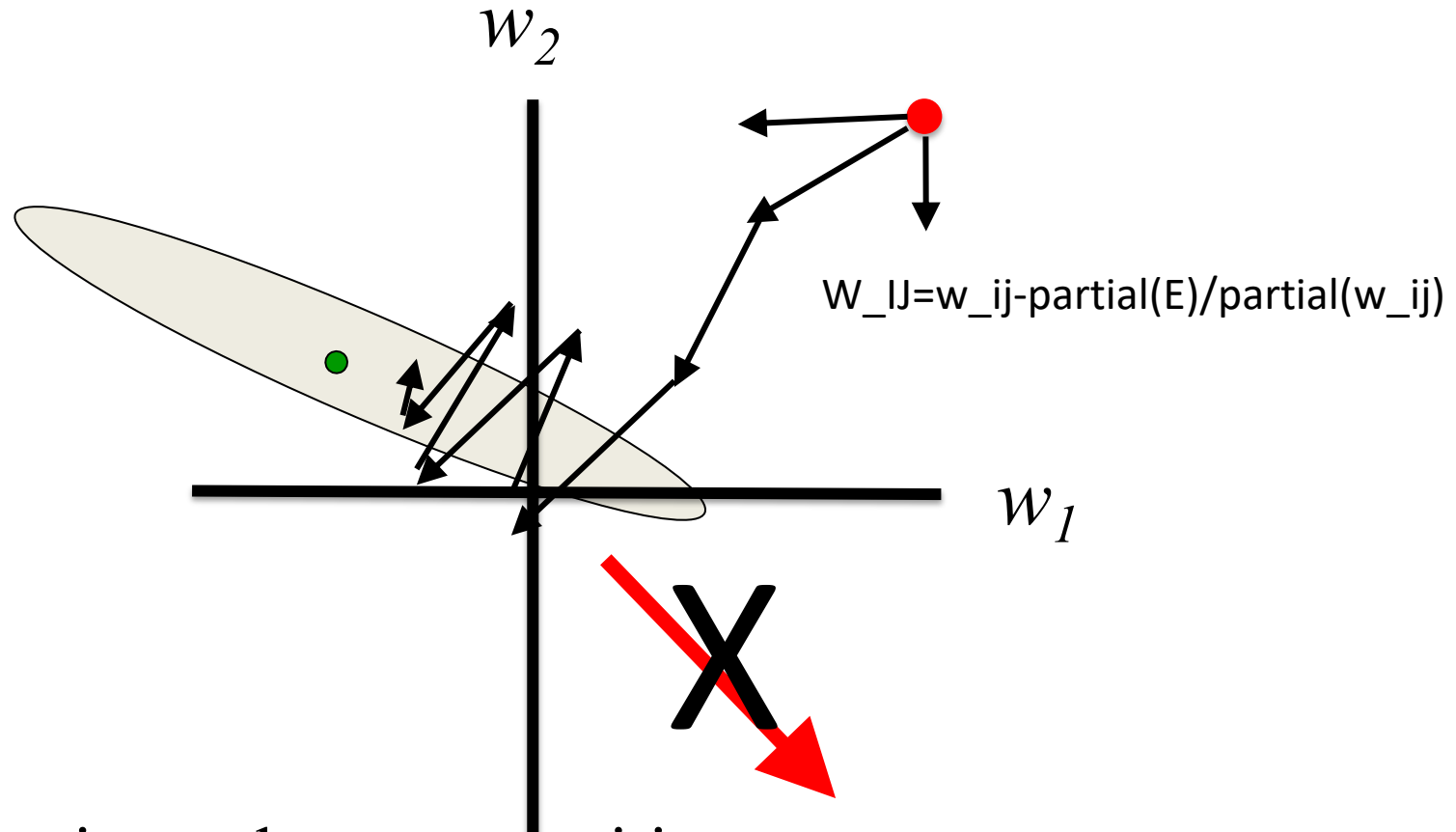
If the x_i 's are all positive $\rightarrow$

the weight changes are all + or -.



Weight changes all positive or negative: where can the weights go?

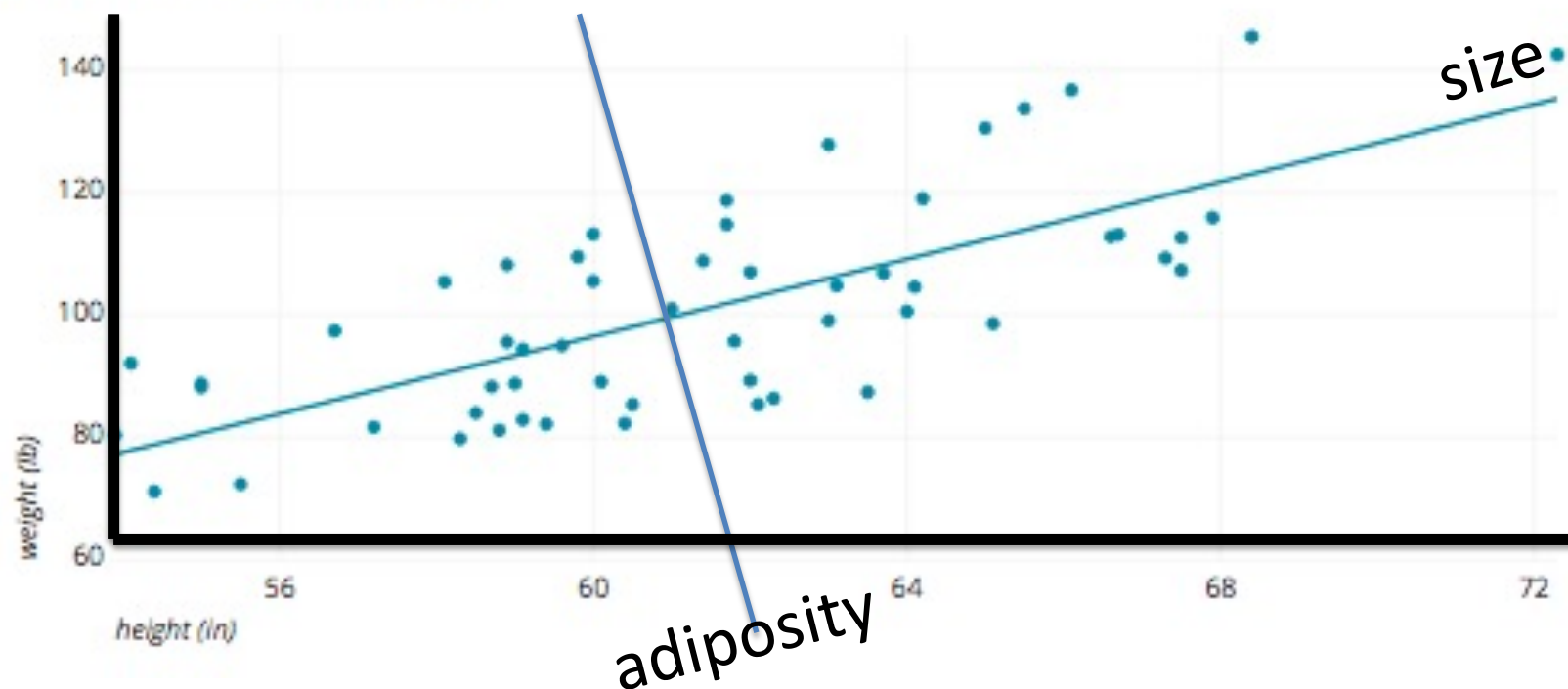
(See previous slide: this is for *one unit's incoming weights*)



- Only changing to be more positive or more negative...never one positive and one negative.

What's wrong with correlated inputs?

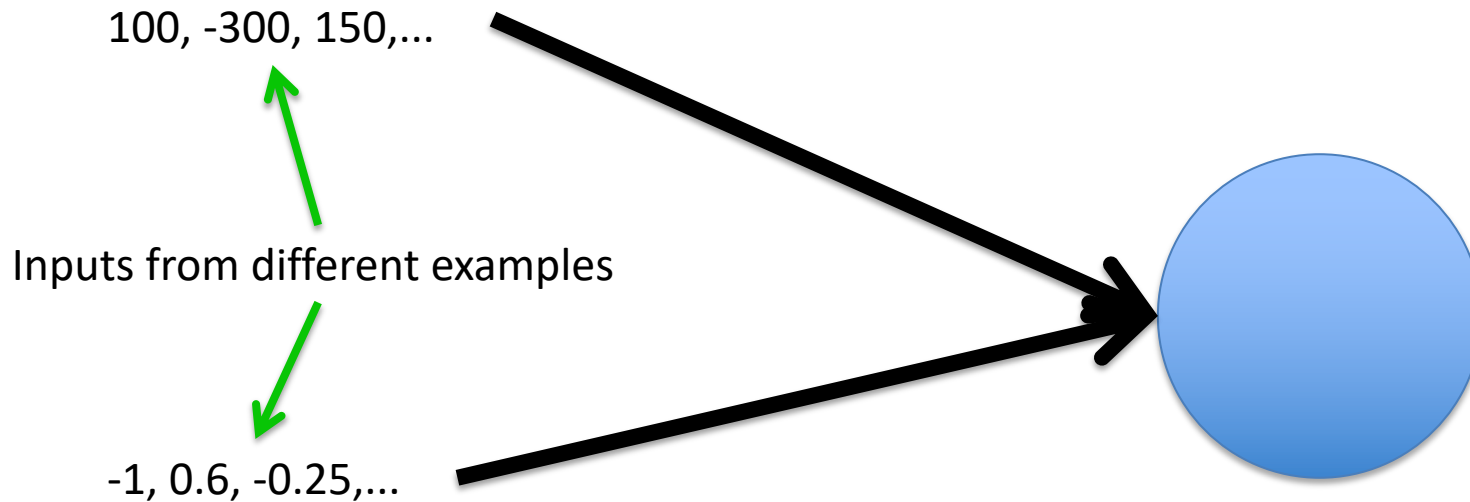
Weight and Height of Children



- These two highly correlated variables encode very similar information: size
- If we did PCA (decorrelated them), we would get two different kinds of information: size and adiposity (fatness)

What's wrong with very different scale of inputs?

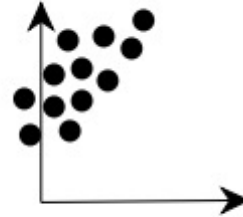
Again, the weight change rule is: $w_{ij} = w_{ij} + \alpha \delta_j x_i$



- The largest weight changes will be for the large inputs (x_i 's).
- If these two inputs are equally important, it will take a long time for the weights to the second input to become big enough to matter...

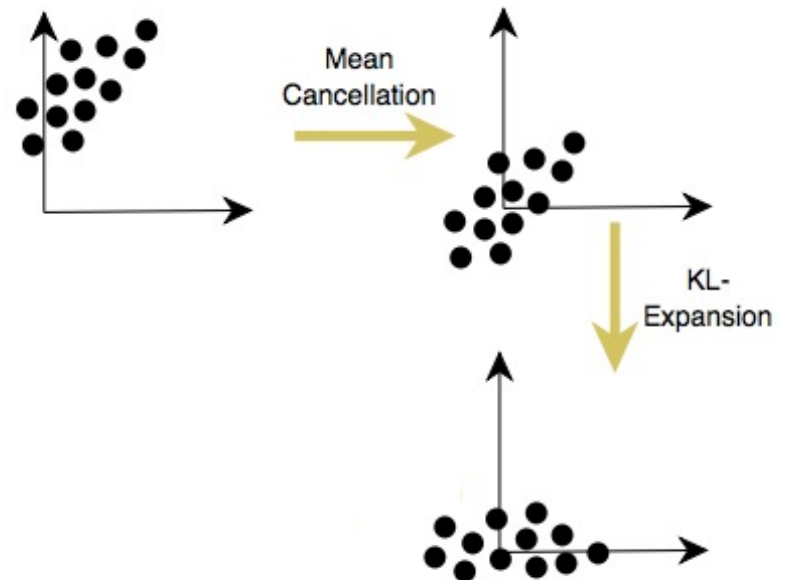
So why PCA of the inputs?

- PCA shifts the mean of the input variables to be 0 – not all positive or negative!



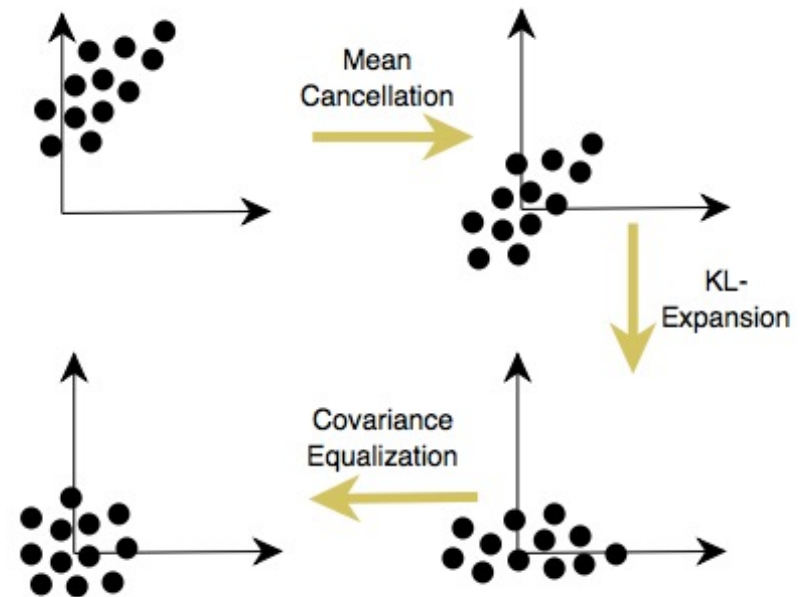
So why PCA of the inputs?

- PCA shifts the mean of the input variables to be 0 – not all positive or negative!
- PCA *decorrelates* the inputs
- And...you can throw away the dimensions with the smallest eigenvalues – achieving dimensionality reduction.



So why PCA of the inputs?

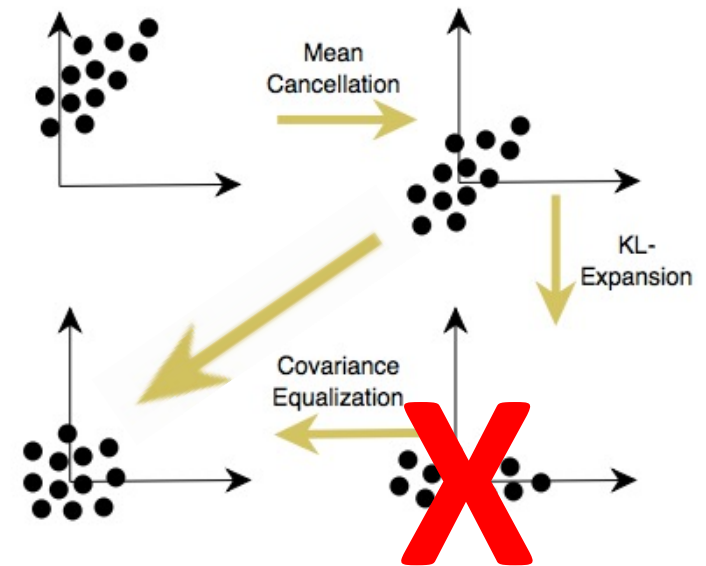
- PCA shifts the mean of the input variables to be 0 – not all positive or negative!
- PCA *decorrelates* the inputs
- And...you can throw away the dimensions with the smallest eigenvalues – achieving dimensionality reduction.
- If you then divide by the standard deviation, makes them all of equal “size” – **this last step is not part of PCA, but it is good for neural nets**



(Note: KL-Expansion and the Hotelling Transform are just different names for PCA)

Difference between PCA and z-scoring

- Z-scoring shifts the mean of the input variables to be 0 – not all positive or negative!
- Z-scoring does ***not*** decorrelate the inputs – because it is applied to each variable independently.
- And...you ***cannot*** throw away the dimensions with the smallest eigenvalues –because you don't compute the eigenvectors.
- Z-scoring divides by the standard deviation, making all of the variables 0 mean and unit standard deviation.
- So the variables are all of the same size



Side Note:

A linear autoencoder essentially does PCA

- It's minimizing the same thing: squared error
- A hidden unit is like a principal component, and so the input is projecting onto that is its coordinate on that component
- But the variance is spread across all of them.

This lecture

- Tricks of the Trade:
 - Stochastic gradient descent versus batch learning
 - Shuffling the examples
 - Perform PCA of the inputs
 - *Change the sigmoid*
 - Initialize the weights
 - Adaptive learning rates
 - Momentum

Ok, we've done all these good things

- And now...we put these nicely formatted inputs into the logistic sigmoid????
- What will the activations be?
- Then the next layer up gets all positive inputs again!!!

BAD!!!

We need a different activation fn.

So, change the sigmoid to have the same properties as before - because these are inputs to the *next* layer.

- Standard sigmoid makes all inputs to the next layer positive - BAD.
- a “bipolar” sigmoid - one symmetric around 0 - is better.

Recommended sigmoid:

$$f(x) = 1.7159 * \tanh(0.667x).$$

- Why? If x has unit variance, then the output will have unit variance – the output is “formatted” for the next layer up.
- In particular, this sigmoid has the following properties:
 1. $f(\pm 1) = \pm 1$.
 2. second derivative is max at $x=0$, so the slope is changing fastest there.
 3. effective gain is 1 over the linear range.

This lecture

- Tricks of the Trade:
 - Stochastic gradient descent versus batch learning
 - Shuffling the examples
 - Perform PCA of the inputs
 - Change the sigmoid
 - *Initialize the weights*
 - Momentum
 - Adaptive learning rates

Weight initialization

Initialize the weights so that the properties achieved above are achieved for the next level up.

Need them to be random, not zero (why?)

Too small: no gradient

Too large: no gradient (with a saturating nonlinearity like logistic or tanh)

Weight initialization

Why can't the weights start at zero?

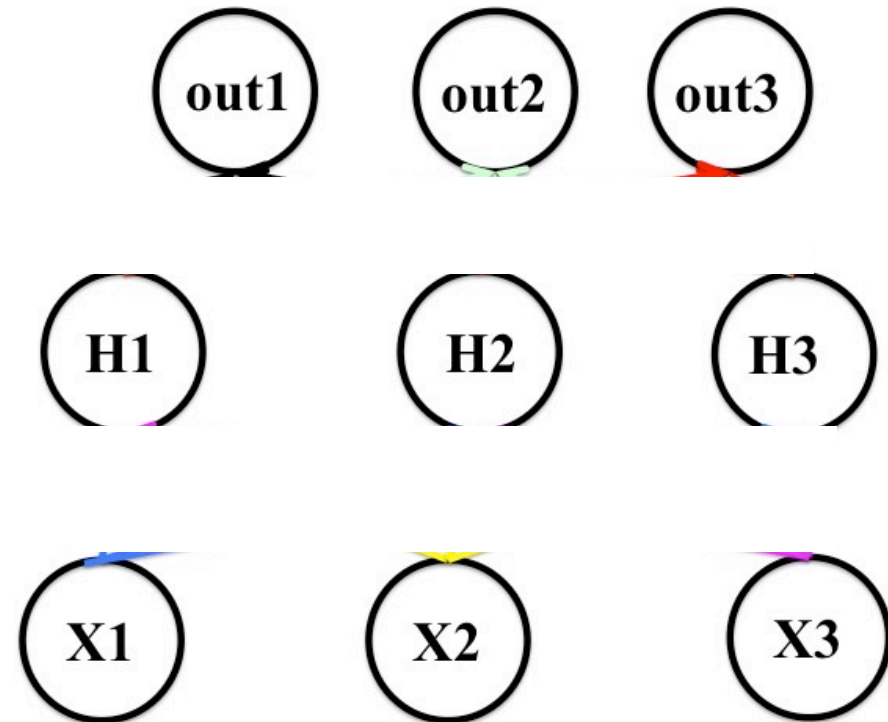
Recall: $w_{ij} = w_{ij} + \alpha \delta_j z_i$

Assume a logistic
hidden unit function:

$\sigma(0) = 0.5$ (all same)

Different outputs will
have different targets

And thus different δ 's



Weight initialization

Why can't the weights start at zero?

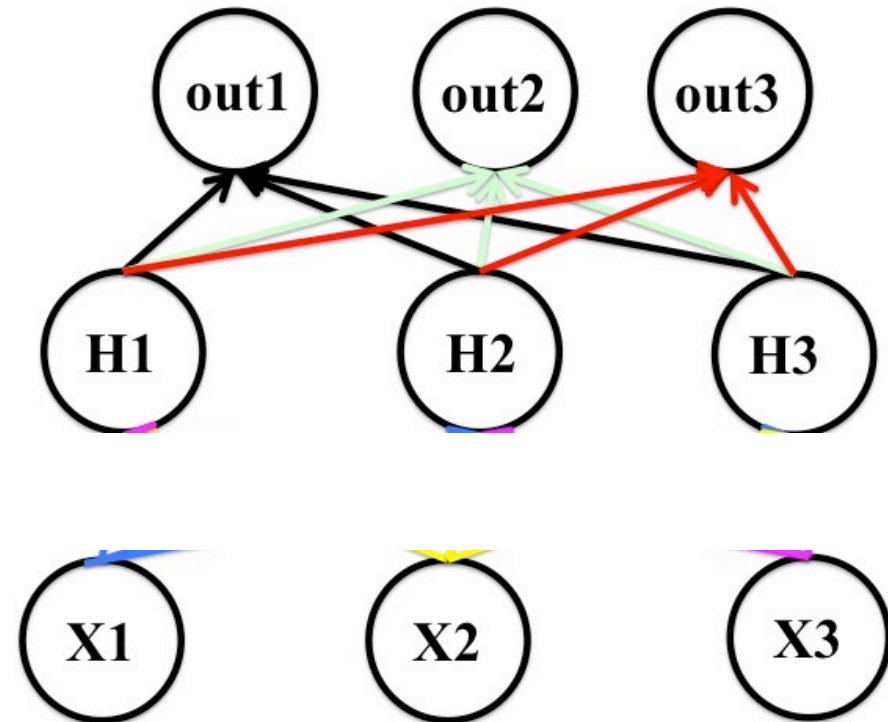
Recall: $w_{ij} = w_{ij} + \alpha \delta_j x_i$

Assume a logistic
hidden unit function

So the *incoming* activations
from each hidden

will be the same...and

The δ 's of the
hiddens will all be
the same...



Weight initialization

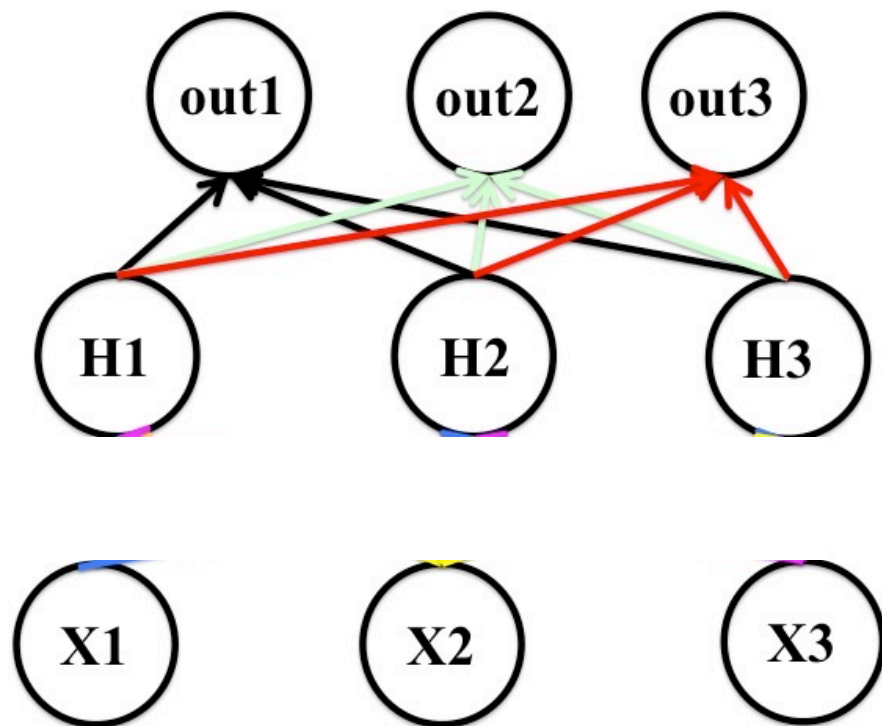
Why can't the weights start at zero?

Recall: $w_{ij} = w_{ij} + \alpha \delta_j x_i$

Assume a logistic
hidden unit function

The δ 's of the
hiddens will all be
the same...

Inputs will be
different...



Weight initialization

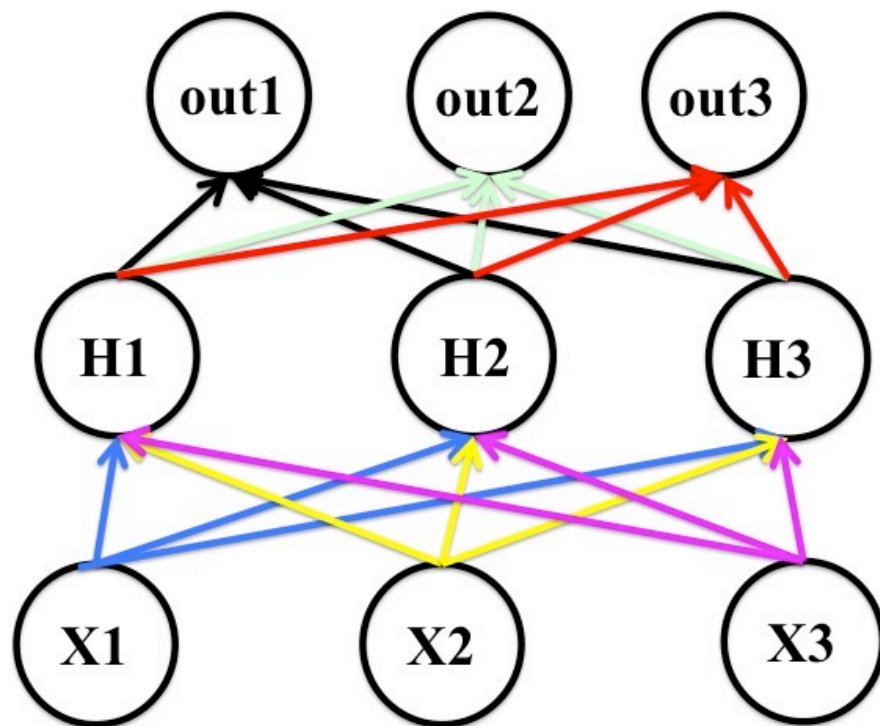
Why can't the weights start at zero?

Recall: $w_{ij} = w_{ij} + \alpha \delta_j x_i$

Assume a logistic
hidden unit function

So outgoing weights
from each input
will be the same...
...and the hidden units
will all compute the same
feature!

Note: If you start with tanh,
all weights will stay zero



So, we start with *random* weights,
which breaks this symmetry

Weight initialization

Ok, the weights will be random – but what should they be?

We want the weighted sum of the inputs (a_j) to be in the linear range of the sigmoid, because:

1. gradients will be largest and
2. the network can learn any linear part of the mapping before the non-linear part.

So, we want the a_j to be 0 mean and unit standard deviation (with the recommended sigmoid)

Recall that one reason we picked: $f(a_j) = 1.7159 * \tanh(0.667a_j)$. is because if a_j has unit variance, then $f(a_j)$ will have unit variance – the output is “formatted” for the next layer up.

Weight initialization

Ok, the weights will be random – but what should they be?

We want the weighted sum of the inputs (a_j) to be in the linear range of the sigmoid, because:

1. gradients will be largest and
2. the network can learn any linear part of the mapping before the non-linear part.

So, we want the a_j to be 0 mean and unit standard deviation (with the recommended sigmoid)

Weight initialization needs to be coordinated with

- a. the input normalization
- b the choice of sigmoid.

So, we have achieved 0 mean and unit standard deviation with the inputs (PCA did this) – we **also** want this to be true for the outputs of the first hidden layer, etc.

Weight initialization

If the x_i are mean 0 and unit standard deviation, and the w_{ij} have mean 0, then on average, the standard deviation of $a_j = XW$ will be (assuming X and W are independent, and $\mathbf{E}$ means expectation and Var means “variance”):

$$\begin{aligned}\text{Var}(XW) &= \text{Var}(X)\text{Var}(W) + \text{Var}(X)(\mathbf{E}(W))^2 + \text{Var}(W)(\mathbf{E}(X))^2 \\ &= 1 * \text{Var}(W) + \text{Var}(X) * 0 + \text{Var}(W) * 0^2 \quad (X, W \text{ are mean } 0, \\ &= \text{Var}(W). \quad \text{and } X \text{ has unit } \sigma)\end{aligned}$$

So, the standard deviation of $a_j = \text{sqrt}(\text{Var}(W))$
 $= \text{sqrt}(\sum_i (w_{ij})^2)$
(because mean is 0)

Weight initialization

Ok, so we've figured out that: $\sigma(a_j) = \sqrt{\sum_i w_{ij}^2}$

Given the “funny tanh“ activation function, we want this number to be 1 because that will make the standard deviation of the input to the *next* layer up have standard deviation 1.

Suppose the number of inputs to unit j is **m** (we call this the ***fan-in to unit***)

So, if we set the standard deviation of the w 's to be $1/(m)^{1/2}$, we will

get: $\sigma(a_j) = \sqrt{\sum_i \frac{1}{\sqrt{m}}^2} = \sqrt{\sum_i \frac{1}{m}} = 1$ (since there are m weights)

Weight initialization

SO, assuming that:

1. The inputs have been normalized to 0 mean and unit standard deviation, and
2. The sigmoid is $f(x) = 1.7159 * \tanh(2/3x)$,

THEN:

The weights w_{ij} into a node j should be drawn from a distribution with 0 mean and standard deviation $= (1/m)^{1/2}$,

where m is the fan-in to node j .

Weight initialization

The point being, that after normalizing the inputs via PCA, by combining the particular weight initialization with the right sigmoid, we get normalized inputs to the *next* layer up!

Note: This makes intuitive sense: If a hidden unit has a big fan-in, small changes on many of its incoming weights can cause the learning to overshoot.

- We generally want smaller incoming weights when the fan-in is big, and vice-versa, so initialize the weights to be proportional to $1/\sqrt{\text{fan-in}}$.

Note!

- Obviously, ReLU units don't have the same properties!
- Need a different analysis for these.
- Xavier initialization (Glorot & Bengio, 2010):

- Where n_j and n_{j+1} are the fan-in and fan-out of a unit, respectively)

$$W \sim U\left[-\frac{\sqrt{6}}{\sqrt{n_j + n_{j+1}}}, \frac{\sqrt{6}}{\sqrt{n_j + n_{j+1}}}\right] \quad (16)$$

- Better: Kaiming initialization (He, et al., 2015):

$$W \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_l}}\right), \text{ where } n_l \text{ is the fan-in, and}$$

Here, $\mathcal{N}$ denotes a normal distribution

Batch Normalization

But all this careful work won't survive all the weight changes during learning...things will no longer be zero mean and unit s.d.

...enter *batch normalization*.

Batch normalization uses all of these insights – but *dynamically*, as the network is being trained.

33,522 Citations!!!

(as of 12:31PM 01/23/2022)

Now, 52,675 Citations!!!

(as of 1:48PM, 01/18/2024)

(4 more as of an hour ago...)



Batch Normalization

Batch normalization normalizes *all of the inputs in the network – the inputs to each hidden layer throughout the network, on a per unit basis, over each minibatch.*

Batch Normalization:

The *Cliff Notes* version...

Batch normalization takes a minibatch and

1. z-scores each variable (hidden layer inputs – the a_j 's) – *at every layer of the network individually, over the mini-batch*
2. Then gives the network the chance to *undo* batch normalization by giving it adaptive parameters

Batch Normalization:

The *Cliff Notes* version...

Batch normalization takes a minibatch and

1. z-scores each input variable over the mini-batch

$$\hat{a}_i = \frac{a_i - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}}$$

where μ_i and σ_i^2 are the mean and variance of a_i over the mini-batch.

Again – a_i is the net input to a unit *at any layer of the network* (But note – this is done within a layer)

Batch Normalization:

The *Cliff Notes* version...

Batch normalization takes a minibatch and

2. Allows the network to learn to *undo this if necessary*:

$$\tilde{a}_i = \gamma_i \hat{a}_i + \beta_i$$

where γ_i and β_i are learnable parameters

So, the original a_i is replaced by $\tilde{a}_i$

And this whole transform can be back-propagated through...because it is all differentiable.

Batch Normalization: Summary

(only the variable names have been changed to protect the innocent)

Input: Values of a over a mini-batch: $\mathcal{B} = \{a_{1\dots m}\};$

Parameters to be learned: γ, β

Output: $\{\tilde{a}_i = \text{BN}_{\gamma,\beta}(a_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m a_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (a_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{a}_i \leftarrow \frac{a_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$\tilde{a}_i \leftarrow \gamma \hat{a}_i + \beta \equiv \text{BN}_{\gamma,\beta}(a_i) \quad // \text{ scale and shift}$$

Algorithm 1: Batch Normalizing Transform, applied to activation a over a mini-batch.

Adapting the learning rate & Momentum: (Mostly) Hinton Slides...

This lecture

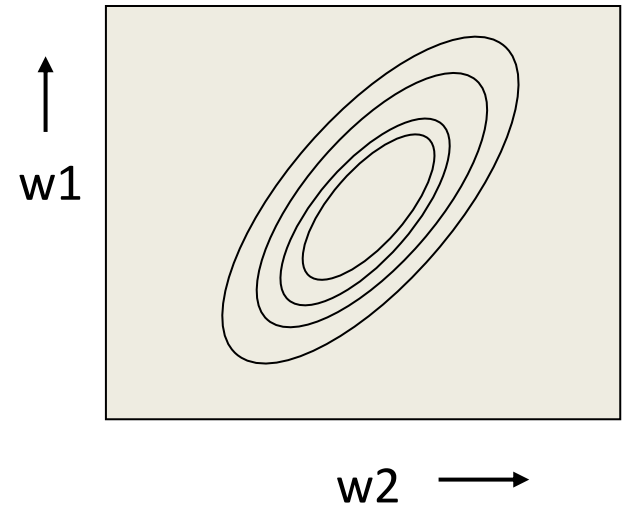
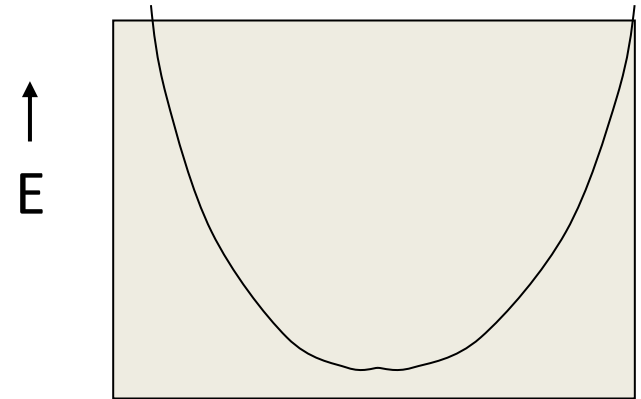
- Tricks of the Trade:
 - Stochastic gradient descent versus batch learning
 - Shuffling the examples
 - Perform PCA of the inputs
 - Change the sigmoid
 - Initialize the weights
 - *Momentum*
 - Adaptive learning rates

Ways to speed up mini-batch learning

- Use “momentum”
 - Instead of using the gradient to change the weights, keep a *running average* of the weight changes.
- Use separate adaptive learning rates for *each* parameter
 - Slowly adjust the rate using the consistency of the gradient for that parameter.
- Geoff Hinton’s idea:
 - rmsprop**: Divide the learning rate for a weight by a running average of the magnitudes of recent gradients for that weight.
 - This normalizes the learning rate...
- These ideas will be explained in the following slides...after some intuition...

Reminder: The error surface for a linear neuron (using MSE)

- The error surface lies in a space with a horizontal axis for each weight and one vertical axis for the error.
 - For a linear neuron (with squared error), it is a quadratic bowl.
 - Vertical cross-sections are parabolas.
 - Horizontal cross-sections are ellipses.
- For multi-layer, non-linear nets the error surface is *much* more complicated – with local minima, long plateaus, saddle points, etc.
 - But locally, a piece of a quadratic bowl is usually a very good approximation.



Clicker Question

Convergence with a multi-layer, non-linear network indicates that the network has found a local minima along the error surface.

A. True

B. False

Clicker Question

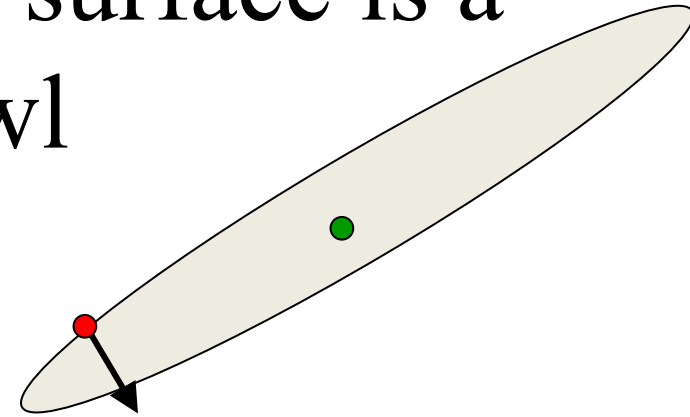
Convergence with a multi-layer, non-linear network indicates that the network has found a local minima along the error surface.

A. True

B. False

Convergence speed of full batch learning when the error surface is a quadratic bowl

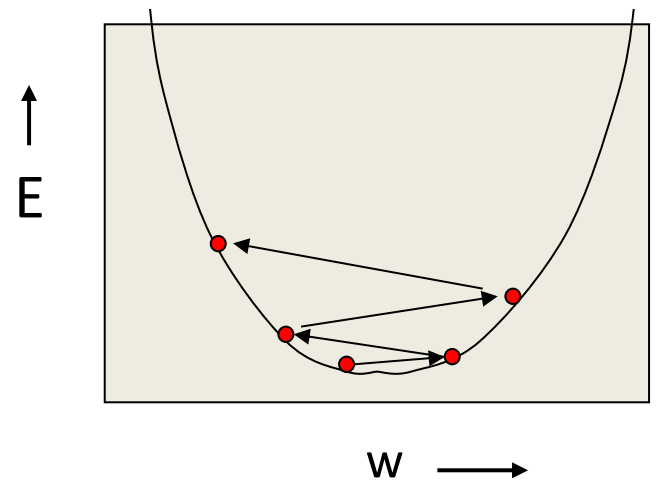
- Going downhill reduces the error, but the direction of steepest descent does not point at the minimum *unless the ellipse is a circle*.
 - The gradient is big in the direction in which we only want to travel a small distance.
 - The gradient is small in the direction in which we want to travel a large distance.



Even for non-linear multi-layer nets, the error surface is locally quadratic, so the same speed issues apply.

How the learning goes wrong

- If the learning rate is big, the weights sash to and fro across the ravine.
 - If the learning rate is too big, this oscillation diverges.
- What we would like to achieve:
 - Move quickly in directions with small but consistent gradients.
 - Move slowly in directions with big but inconsistent gradients.

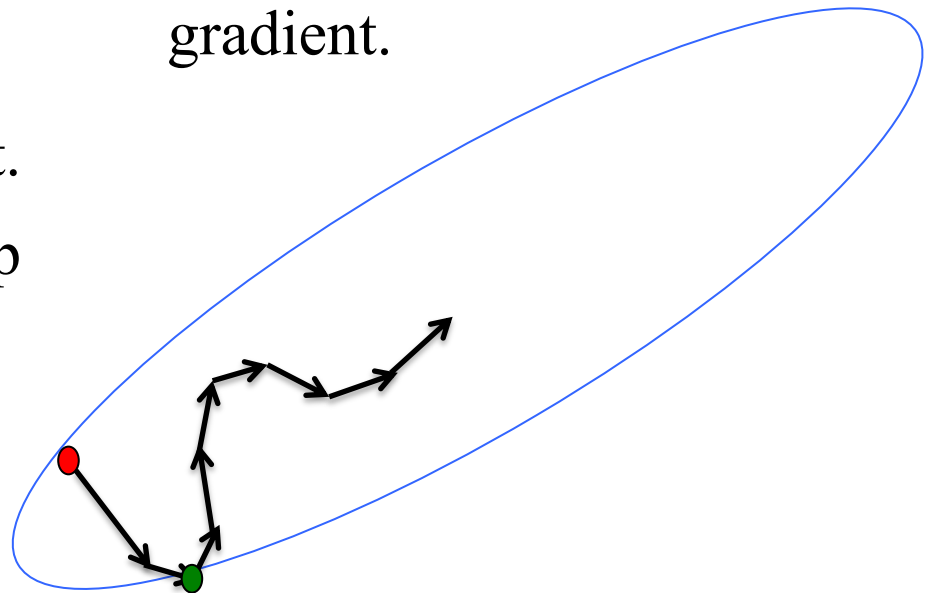


The intuition behind the momentum method

Imagine a ball on the error surface. The location of the ball in the horizontal plane represents the weight vector.

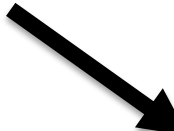
- The ball starts off by following the gradient, but once it has velocity, it no longer does steepest descent.
- Its momentum makes it keep going in the previous direction.

- Momentum damps oscillations in directions of high curvature by combining gradients with opposite signs.
- It builds up speed in directions with a gentle but consistent gradient.



The equation of the momentum method

The effect of the gradient at time t is to increment the previous average.


$$\Delta \mathbf{w}(t) = \boxed{\gamma} \Delta \mathbf{w}(t - 1) - \alpha \frac{\partial E}{\partial \mathbf{w}}(t)$$

The average also decays by γ which is slightly less than 1.

Thus, the weight change is the sum of the previous weight change and the new gradient.

The behavior of the momentum method

- If the error surface is a tilted plane, the ball reaches a terminal velocity.
 - If the momentum is close to 1, this is much faster than simple gradient descent.
 - A tilted plane means the gradient is constant and we can solve the recurrence relation:
- Note: If the momentum weight, γ , is 0.9, then this means

$$\Delta \mathbf{w}(\infty) = 10 * \left(-\alpha \frac{\partial E}{\partial \mathbf{w}} \right)$$

- I.e., we are learning *10 times faster than our original learning rate.*

$$\Delta \mathbf{w}(\infty) = \frac{1}{1-\gamma} \left(-\alpha \frac{\partial E}{\partial \mathbf{w}} \right) \text{ (again, here we are assuming that } -\alpha \frac{\partial E}{\partial \mathbf{w}} \text{ is a constant)}$$

An Aside – solving the recurrence relation

- What we mean by “solving the recurrence relation” is, what does this iteration converge to, i.e., when does the update not change?
- I.e., when does $\Delta \mathbf{w}(t) = \Delta \mathbf{w}(t - 1)$?
- To solve this, we *assume* $\Delta \mathbf{w}(t) = \Delta \mathbf{w}(t - 1)$ and solve it.
- We’ll write this as $\Delta \mathbf{w}(\infty)$

$$\Delta \mathbf{w}(\infty) = \gamma \Delta \mathbf{w}(\infty) \alpha \frac{\partial E}{\partial \mathbf{w}}(\infty) \rightarrow$$

$$(1 - \gamma) \Delta \mathbf{w}(\infty) = -\alpha \frac{\partial E}{\partial \mathbf{w}} \rightarrow$$

$$\Delta \mathbf{w}(\infty) = \frac{1}{(1-\gamma)} \left(-\alpha \frac{\partial E}{\partial \mathbf{w}} \right)$$

Using momentum: practical advice from Geoff

- At the beginning of learning there may be very large (and inconsistent) gradients.
 - So, start with a small momentum (e.g., 0.5).
 - Once the large gradients have disappeared and the weights are stuck in a ravine the momentum can be smoothly raised to its final value (e.g., 0.9 or even 0.99)
- We can learn at a rate that would oscillate without the momentum.

Advice from Gary:

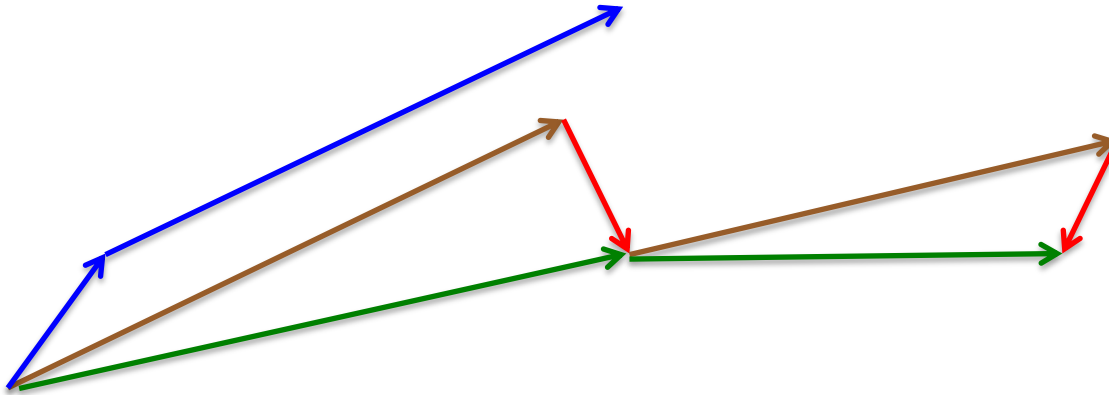
Ignore Geoff's advice and just start with $\gamma = 0.9$ or 0.99

A better type of momentum (Nesterov 1983)

- The standard momentum method **first** computes the gradient at the current location (at step t) and **then** takes a big jump in the direction of the updated accumulated gradient.
- Ilya Sutskever suggested a new form of momentum that often works better.
 - Inspired by the Nesterov method for optimizing convex functions.
- **First** make a big jump in the direction of the previous accumulated gradient (from step $t-1$)
- **Then** measure the gradient where you end up and make a correction.
 - Its better to correct a mistake **after** you have made it!

A picture of the Nesterov method

- **First** make a big jump in the direction of the previous accumulated gradient.
- **Then** measure the gradient where you end up and make a correction.



brown vector = jump,
accumulated gradient

red vector = correction,

green vector =

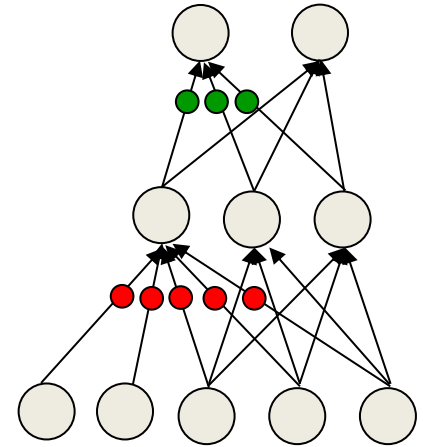
blue vectors = standard momentum

This lecture

- Tricks of the Trade:
 - Stochastic gradient descent versus batch learning
 - Shuffling the examples
 - Perform PCA of the inputs
 - Change the sigmoid
 - Initialize the weights
 - Momentum
 - *Adaptive learning rates*

The intuition behind separate adaptive learning rates

- In a multilayer net, the appropriate learning rates can vary widely between weights:
 - The magnitudes of the gradients are often very different for different layers, especially if the initial weights are small.
 - The fan-in of a unit determines the size of the “overshoot” effects caused by simultaneously changing many of the incoming weights of a unit to correct the same error.

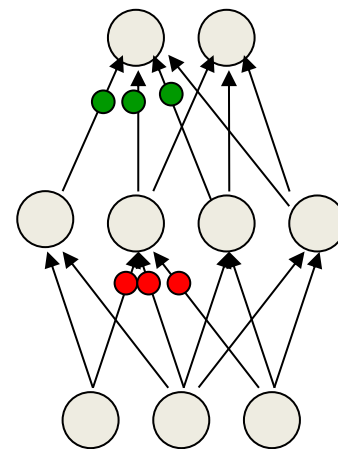


Gradients can get very small in the early layers of very deep nets.

The fan-in often varies widely between layers.

The intuition behind separate adaptive learning rates

- In a multilayer net, the appropriate learning rates can vary widely between weights:
- So use a global learning rate (set by hand) multiplied by an appropriate local gain that is determined empirically for each weight.



Gradients can get very small in the early layers of very deep nets.

The fan-in often varies widely between layers.

One way to determine the individual learning rates

- Start with a local gain of 1 for every weight ($g_{ij}=1$)
- Increase the local gain if the gradient for that weight does not change sign.
- Decrease the local gain if the gradient for that weight is changing sign

$$\Delta w_{ij} = -\varepsilon g_{ij} \frac{\partial E}{\partial w_{ij}}$$

$$\text{if } \left(\frac{\partial E}{\partial w_{ij}}(t) \frac{\partial E}{\partial w_{ij}}(t-1) \right) > 0$$

$$\text{then } g_{ij}(t) = g_{ij}(t-1) + .05$$

$$\text{else } g_{ij}(t) = g_{ij}(t-1) * .95$$

One way to determine the individual learning rates

- Use small additive increases and multiplicative decreases (for mini-batch)

$$\Delta w_{ij} = -\varepsilon g_{ij} \frac{\partial E}{\partial w_{ij}}$$

– This ensures that big gains decay rapidly when oscillations start.

– If the gradient is totally random the gain will hover around 1 when we increase **plus** δ half the time and decrease by **times** $1 - \delta$ half the time (here δ is .05)

(you can try this at home!)

$$\text{if } \left(\frac{\partial E}{\partial w_{ij}}(t) \frac{\partial E}{\partial w_{ij}}(t-1) \right) > 0$$

$$\text{by then } g_{ij}(t) = g_{ij}(t-1) + .05$$

$$\text{else } g_{ij}(t) = g_{ij}(t-1) * .95$$

Tricks for making adaptive learning rates work better

- Limit the gains to lie in some reasonable range
 - e.g. $[0.1, 10]$ or $[.01, 100]$
- Use full batch learning or big mini-batches
 - This ensures that changes in the sign of the gradient are not mainly due to the sampling error of a mini-batch.
- Adaptive learning rates can be combined with momentum.
 - Use the agreement in sign between the current gradient for a weight and the *momentum* for that weight (Jacobs, 1989).
- Adaptive learning rates only deal with axis-aligned effects.
 - Momentum does not care about the alignment of the axes.

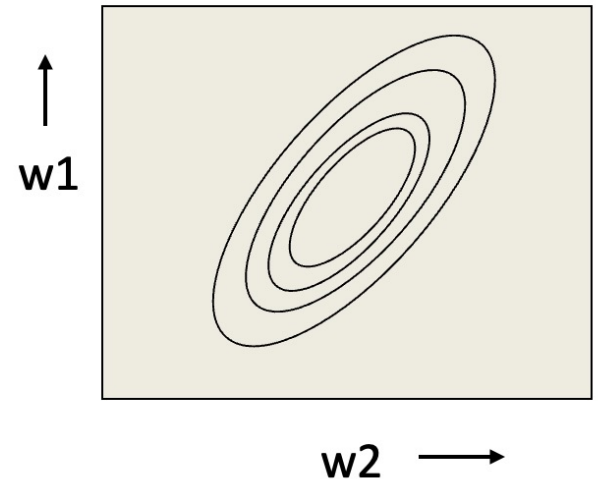
Tricks for making adaptive learning rates work better

- Adaptive learning rates only deal with axis-aligned effects.
 - Momentum does not care about the alignment of the axes

What does “axis-aligned effects” mean?

Answer: Adaptive learning rates affect individual weights – they change the learning rate along that axis

Momentum doesn't.



rprop: Using only the sign of the gradient

- Short for *resilient backpropagation*
- A learning heuristic for supervised learning created by Martin Riedmiller and Heinrich Braun in 1992.
- Motivation: The magnitude of the gradient can be very different for different weights and can change during learning.
 - This makes it hard to choose a single global learning rate.
- For **full batch learning**, we can deal with this variation by *only using the sign of the gradient*.
 - The weight updates are all the same magnitude.
 - This escapes from plateaus with tiny gradients quickly.

rprop: Using only the sign of the gradient

- Rprop combines the idea of only using the sign of the gradient with the idea of adapting the step size separately for each weight.
 - Increase the step size for a weight **multiplicatively** (*e.g., times 1.2*) if the signs of its last two gradients agree.
 - Otherwise decrease the step size multiplicatively (*e.g., times 0.5*).
 - Limit the step sizes to be less than 50 and more than a millionth
- *Again*, this is a heuristic algorithm, and only applies to full batch learning.

Why rprop does not work with mini-batches

- The idea behind stochastic gradient descent is that when the learning rate is small, it averages the gradients over successive mini-batches.
 - Consider a weight that gets a gradient of $+0.1$ on nine mini-batches and a gradient of -0.9 on the tenth mini-batch.
 - We want this weight to stay roughly where it is.

Why rprop does not work with mini-batches

- rprop would increment the weight nine times ($+1*9$) and decrement it once by about the same amount ($-1*1$) (assuming any adaptation of the step sizes is small on this time-scale).
 - So the weight would grow a lot ($+8$).
- Is there a way to combine:
 - The robustness of rprop.
 - The efficiency of mini-batches.
 - The effective averaging of gradients over mini-batches?

rmsprop: A mini-batch version of rprop

- rprop is equivalent to using the gradient but also dividing by the size of the gradient. (Why?)
 - Why: $\text{sign}(x) = x/|x|$
 - The problem with mini-batch rprop is that we divide by a different number for each mini-batch.
 - So why not force the number we divide by to be very similar for adjacent mini-batches?
 - Idea: Keep a moving average of the squared gradient for each weight

rmsprop: A mini-batch version of rprop

- rmsprop: Keep a moving average of the squared gradient for each weight

$$MeanSquare(w, t) = 0.9 \text{ MeanSquare}(w, t-1) + 0.1 \left(\frac{\partial E}{\partial w}(t) \right)^2$$

- Dividing the gradient by $\sqrt{MeanSquare(w, t)}$ makes the learning work much better (Tijmen Tieleman).

Summary of learning methods for neural networks

- For small datasets (e.g. 10,000 cases) or bigger datasets without much redundancy, use a full-batch method.
 - Conjugate gradient, LBFGS ...
 - adaptive learning rates, rprop ...
- For big, redundant datasets use mini-batches.
 - Try gradient descent with momentum.
 - Try the adam optimizer
 - Try whatever Yann LeCun is doing at the moment!

- Why there is no simple recipe:

Neural nets differ a lot:

- Very deep nets (especially ones with narrow bottlenecks).
- Recurrent nets.
- Wide shallow nets.

Tasks differ a lot:

- Some require very accurate weights, some don't.
- Some have many very rare cases (e.g. words).